GATE CS Master Sheet: SQL NULL Semantics & 3VL

Complete Logic, Mental Shortcuts, Aggregates & Subquery Traps

GATE CSE REVISION

1. The Mental Shortcut for 3VL (Three-Valued Logic)

Never memorize bulky truth tables. Convert SQL logic to numbers:

> **TRUE = 1.0 | UNKNOWN (U) = 0.5 | FALSE = 0.0**

- **AND is min(A, B):**
  `TRUE AND U` → min(1, 0.5) = **0.5 (UNKNOWN)**
  `FALSE AND U` → min(0, 0.5) = **0 (FALSE)**
- **OR is max(A, B):**
  `TRUE OR U` → max(1, 0.5) = **1 (TRUE)**
  `FALSE OR U` → max(0, 0.5) = **0.5 (UNKNOWN)**
- **NOT is (1 - A):**
  `NOT(TRUE)` = 1 - 1 = **0 (FALSE)**
  `NOT(UNKNOWN)` = 1 - 0.5 = **0.5 (UNKNOWN)**

The Golden WHERE Clause Rule:
A row is selected **ONLY IF** the condition evaluates to **TRUE (1)**.
If it evaluates to **FALSE (0)** or **UNKNOWN (0.5)** → **DISCARDED!**

2. Aggregates with NULL (Counting & Math)

NULL is *missing information*, not 0 or blank.

Aggregate	What it does	All-NULL Group
`COUNT(*)`	Counts total rows (including NULLs)	Row count
`COUNT(A)`	Counts **non-NULL** values only	**0**
`SUM(A)`	Sums non-NULL values	**NULL** (not 0!)
`AVG(A)`	`SUM(A) / COUNT(A)` (divides by non-NULLs!)	**NULL**
`MIN / MAX`	Ignores NULLs completely	**NULL**

Key Functions:

- `COALESCE(x, y, ...)` : Returns 1st non-NULL (e.g. `COALESCE(NULL, 0) = 0`).
- `NULLIF(x, y)` : Returns `NULL` if `x = y`, else returns `x`.

3. The "IS" Operator Family (State Inspector vs Value Comparator)

Why `=` fails on NULL:
`=` checks value equality. Since NULL is unknown, `x = NULL` and `NULL = NULL` both yield **UNKNOWN (0.5)**! It never survives `WHERE`.

Why `IS` works:
`IS` does not compare values; it inspects the *state of the container*. `IS` **NEVER yields UNKNOWN** (strictly TRUE or FALSE)!

Expression	When x = 10	When x = NULL
`x = NULL`	UNKNOWN (0.5)	UNKNOWN (0.5) ×
`x IS NULL`	FALSE (0)	**TRUE (1)** ✓
`x IS NOT NULL`	**TRUE (1)** ✓	FALSE (0) ×

Advanced 3VL State Inspectors:

- `(expr) IS UNKNOWN` : Evaluates to **TRUE** if `expr` yielded UNKNOWN (0.5).
  Example: If `score` is NULL, `(score > 50) IS UNKNOWN` → **TRUE**!
- `(expr) IS TRUE` / `(expr) IS FALSE` : Strictly tests boolean outcome.
- `A IS NOT DISTINCT FROM B` :
  Treats two NULLs as identical!
  - `5 IS NOT DISTINCT FROM 5` → **TRUE**
  - `NULL IS NOT DISTINCT FROM NULL` → **TRUE** (unlike `=`!)
  - `5 IS NOT DISTINCT FROM NULL` → **FALSE**

4. IN vs NOT IN (The Infamous "Zero Rows" GATE Trap)

How `IN` expands (uses OR / max):

`x IN (v1, v2, v3) ≡ (x = v1) OR (x = v2) OR (x = v3)`

- If any match is found, that term is **TRUE (1)**. By the max rule, whole expression is **TRUE**!
- Even if list has NULL, e.g. `10 IN (10, NULL)` → `1 OR 0.5 = 1 (TRUE)`.
- **Rule:** `IN` is safe as long as an exact match exists.

How `NOT IN` expands (uses AND / min):

`x NOT IN (v1, v2, v3) ≡ (x ≠ v1) AND (x ≠ v2) AND (x ≠ v3)`

- What if the list has a NULL? E.g.: `x NOT IN (10, 20, NULL)` `(x ≠ 10) AND (x ≠ 20) AND (x ≠ NULL)`
- The term `(x ≠ NULL)` is **always UNKNOWN (0.5)**!
- By min rule: if not matched → min(1, 1, 0.5) = **0.5 (UNKNOWN)**; if matched → min(0, 1, 0.5) = **0 (FALSE)**.
- **THE GOLDEN TRAP:** `WHERE x NOT IN (..., NULL)` **NEVER evaluates to TRUE → ALWAYS RETURNS 0 ROWS!**

GATE CS Master Sheet: SQL NULL Semantics & 3VL (Contd.)

Subquery Predicates, Quantifiers & Worked Exam Traces

GATE CSE REVISION

5. Quantifiers: ANY (SOME) vs ALL

Used to compare a single value against a set returned by a subquery:

Operator	What it expands to	Intuition / Equivalent	Handling NULL in Subquery
`= ANY ( ... )`	`(x=v1) OR (x=v2) ...`	**Same as `IN`**	Returns TRUE if at least 1 value matches
`<> ALL ( ... )`	`(x≠v1) AND (x≠v2) ...`	**Same as `NOT IN`**	**Returns 0 rows if NULL is present!**
`> ANY ( ... )`	`(x>v1) OR (x>v2) ...`	`> MIN( ... )`	Safe if greater than any non-NULL number
`> ALL ( ... )`	`(x>v1) AND (x>v2) ...`	`> MAX( ... )`	Yields UNKNOWN if NULL is present & no false
`< ANY ( ... )`	`(x<v1) OR (x<v2) ...`	`< MAX( ... )`	Safe if smaller than any non-NULL number
`< ALL ( ... )`	`(x<v1) AND (x<v2) ...`	`< MIN( ... )`	Yields UNKNOWN if NULL is present & no false

The "Empty Subquery" Trap (GATE Favorite):

- `x > ANY (∅)` → **FALSE** (no element exists to satisfy condition).
- `x > ALL (∅)` → **TRUE** (vacuously true: x is greater than every element in an empty set!).

6. EXISTS vs NOT EXISTS (The NULL-Immune Operators)

`EXISTS` only inspects the **count of returned rows**:

- If subquery returns ≥ 1 row → **TRUE (1)**
- If subquery returns 0 rows → **FALSE (0)**
- `EXISTS` **NEVER EVALUATES TO UNKNOWN!**

Even if the subquery returns a row containing only `NULL` (e.g. `SELECT NULL FROM Table`), a row exists! Thus `EXISTS` evaluates to **TRUE**.

Best Practice & GATE Equivalence:

Whenever replacing a subquery that might contain NULLs:

> **AVOID:** `WHERE col NOT IN (SELECT col FROM T)`
> **USE:** `WHERE NOT EXISTS (SELECT 1 FROM T WHERE T.col = Main.col)`

Because `NOT EXISTS` is 100% immune to the NULL trap that destroys `NOT IN`!

7. Master GATE Trace Walkthroughs (Memorize These Scenarios)

Given table `Student(id, marks)` with values: `{40, 60, 80, NULL, 50, NULL}`

Trace 1: Complementary WHERE Query

```
SELECT COUNT(*) FROM Student
WHERE marks ≥ 50 OR NOT(marks ≥ 50);
```

- For non-NULL marks (40, 60, 80, 50): `TRUE OR FALSE = TRUE` → **Survives (4 rows)**.
- For NULL marks: `(NULL ≥ 50) = 0.5`. `NOT(0.5) = 0.5`. `0.5 OR 0.5 = 0.5 (UNKNOWN)` → **DISCARDED!**
- **Result = 4** (Total is 6, but NULL rows never survive either side!).

Trace 2: AVG with NULLs

```
SELECT AVG(marks) FROM Student;
```

- Sum = 40 + 60 + 80 + 50 = 230.
- Non-NULL Count = 4 (NULLs ignored).
- **Result = 230 / 4 = 57.5** (Divided by 4, NOT 6!).

Trace 3: NOT IN with NULL in Subquery

```
SELECT COUNT(*) FROM Student
WHERE marks NOT IN (40, 60, NULL);
```

- Condition: `(marks ≠ 40) AND (marks ≠ 60) AND (marks ≠ NULL)`
- Since `(marks ≠ NULL)` is **0.5**:
  - If marks = 40 → min(0, 1, 0.5) = **0 (FALSE)**.
  - If marks = 80 → min(1, 1, 0.5) = **0.5 (UNKNOWN)**.
- Zero rows evaluate to TRUE!
- **Result = 0**!

Trace 4: EXISTS with NULL row

```
SELECT * FROM Student
WHERE EXISTS (SELECT NULL FROM Student WHERE id = 4);
```

- Row `id = 4` exists (even though marks is NULL).
- Subquery returns 1 row containing NULL.
- Row count = 1 ≥ 1 → `EXISTS` is **TRUE**!